

OpenParEMg User's Manual

Version 1.0

June 2026

Brian Young



Contents

1	Introduction	2
2	Installation and Execution	2
3	GUI Layout	2
3.1	Drawing	2
3.2	Path	2
3.3	Port	3
3.4	Boundary	3
3.5	Mesh	3
4	Drawing Philosophy	3
4.1	Drawing Plane	3
5	Setup	4
5.1	Path Philosophy	4
5.2	Port Creation	4
6	Tutorials	4
A	Drawing File Format Specification	5

1 Introduction

OpenParEMg is a graphical user interface (GUI) front-end for OpenParEM3D creating an integrated environment for setting up and running OpenParEM3D projects. It includes a native 3D drawing package fully customized for electromagnetic simulations, meshing with gmsh[1][2], input forms for all OpenParEM3D settings, and OpenParEM3D execution control. Given the availability of excellent post-processing capability with various open source projects like LibreOffice Calc[3] for S-parameter plotting and ParaView[4] for field visualization, post-processing is not currently integrated into OpenParEMg.

With OpenParEMg, OpenParEM3D remains a completely stand-alone executable. After setting up a project with OpenParEMg, OpenParEM3D can still be run manually at the command line, and projects can be modified at the command line and run with OpenParEM3D standalone or imported back into OpenParEMg and executed there. Existing projects set up with a discrete flow, such as with FreeCAD[?] for drawing, continue to work unchanged.

OpenParEMg continues the design philosophy of relatively simple text-based input files. The drawing file is custom to OpenParEMg and text-based for easy modification and scripting. Given the required programming effort, it is assumed that additional drawing file formats, such as FreeCAD, can be supported for import.

2 Installation and Execution

Installation and execution instructions are provided in a separate document called "Installation_Execution.pdf". The document is available at <https://openparem.org> and at <https://github.com/OpenParEM/OpenParEMCommon> in the doc directory.

3 GUI Layout

The GUI has a menu bar that generally works from left to right as the project proceeds, from drawing, to setup, to execution. There are two main windows: the item tree and the drawing window. The item tree has five major categories: **Drawing**, **Path**, **Port**, **Boundary**, and **Mesh**. Where appropriate, the item tree and drawing window are cross-linked, such that clicking on a drawing item or path highlights the structure in the drawing window, and vice versa. Right-clicked context menus in the item tree and drawing window show the available options for the selected item or drawing structure.

3.1 Drawing

The **Drawing** item tree entry shows every drawing object in a hierarchy of objects created through the various drawing functions. Selecting an item in the **Drawing** hierarchy selects the matching object in the drawing window. There is only one object per hierarchical item. The object may be a 2D primitive, a 3D object created through operations on 2D primitives, or a Brep object. Legal operations on the selected item are provided by a right-click context menu, where limitations are made to prevent corruption of a 3D object or the creation of 3D objects that are not physically sensible.

3.2 Path

Paths are a fundamental concept in OpenParEM3D, where they are used to define ports and integration lines. All paths are kept in the **Path** item to act as a sort of database for further assignment for and within ports and for boundaries.

Paths can be drawn directly when the option is provided in port setup or can be created by first drawing a 2D primitive then using the context menu option **Convert To Path** on the selected primitive.

Paths can be assigned to port voltage and current integration paths by selecting the path then either **voltage** or **current** with the CTRL pressed (i.e. multiple selection), then right-clicking the **voltage** or **current** and selecting **Add Path**. OpenParEMg checks to verify that the path exists within the port.

The linkages between paths, ports, and boundaries are shown by highlighting the path everywhere when it is selected. The selection can take place in **Path** or in **Port** and **Boundary**.

Since paths are linked into ports and boundaries, they cannot be deleted until they are no longer in use. To delete a path, it must first be removed everywhere it is being used.

3.3 Port

S-parameter calculations require at least one port with one mode to be defined. See the documents "OpenParEM3D_Users_Manual.pdf" and "OpenParEM3D_Theory_Methodology_Accuracy.pdf" for detailed discussion on setting up ports. Port setup is the area requiring the most expertise in successfully running OpenParEM3D. Each port has an item entry with additional hierarchical items to complete the port setup. Assigning paths for voltage and/or current integration paths is a required operation for the user.

3.4 Boundary

Paths can be assigned to create boundaries plus boundaries can be created from faces. A project may or may not need boundaries given that there is a default boundary, which is the perfect electric conductor (PEC) unless it is changed by the user. So boundaries are only needed for radiation boundaries and for surface impedance boundaries if different from the default.

3.5 Mesh

The **Mesh** item enables the showing and hiding of various mesh element types once a mesh has been loaded or generated. All mesh operations are performed using the **Mesh** menu.

4 Drawing Philosophy

Drawings always start with a 2D primitive drawn on a drawing plane. The default drawing plane is the x-y plane at $z=0$, and this can be changed several different ways as described in Sec. 4.1. There are four primitives that can be drawn: line, polyline (open), polyline (closed), polycircle, and rectangle. The line and polyline (open) are for integration paths for voltage and currents plus general use as temporary construction lines. The polyline (closed), polycircle, and rectangle are for extrusion to create 3D objects. Once 3D objects are created, they can be merged or subtracted to create more complex 3D objects to complete the drawing of the full structure to be simulated.

Drawing features are controlled by the menu options **Select** and **Draw** plus context-sensitive options from pop-up menus when right-clicking the mouse.

4.1 Drawing Plane

The drawing plane is critical for drawing more complex geometries. All 2D primitives are drawn on the drawing plane. The drawing plane can be seen by selecting **Draw**→**Drawing Plane** ...→**Show**. The plane can be changed to one of the other primary axes, such as with **Set to YZ Plane**. Once a completed 3D object starts to build up, it becomes a necessity to set the drawing plane specific faces on the object, and this can be achieved by selecting **Set to Face** then clicking on the desired face to move the drawing plane to that face.

For setting the drawing plane to a face, there are two options. The first is **Set to Face**, which sets the drawing plane to the face but does not align the drawing axis. For line, polyline, and polycircle drawing primitives, the lack of an axis has no consequence. For rectangle primitives, it can lead to the rectangle being mis-oriented to the 3D object. To enable the drawing of properly aligned rectangles, a second option for setting the drawing plane to a face is **Set to Face with Axis**, which enables the selection of a desired face for the drawing plane followed by a form enabling the selection of two points to define the origin and the x-direction.

The drawing plane can also be equivalently set by first selecting a face followed by the assignment to the drawing plane. To select a face, select **Select→Face**, then select the desired face for the drawing plane, right-click to get the context menu, then choose **Set Drawing Plane** or **Set Drawing Plane with Axis**.

5 Setup

OpenParEM3D requires the setup of ports and integration lines within the ports. In addition, ports required at least one mode each, where each mode becomes on Sport in the computed S-parameters.

5.1 Path Philosophy

Paths defined port locations and integration paths, and they are kept in a path database show in the **Path** item in the item selection tree.

5.2 Port Creation

6 Tutorials

A Drawing File Format Specification

```
#OpenParEMgDrawing 1.0 // required on first real line

// a comment

// Each item represents one physical structure in the drawing and is shown in the item tree.

// 2D primitives

Line
  name=text
  point1=(double,double,double) // (x,y,z) coordinate in m
  point2=(double,double,double) // (x,y,z) coordinate in m
  normal=(double,double,double) // normal to the line (i.e. normal to the drawing plane)
EndLine

// polylines must lie on a plane
// a closed polyline has point[0]=point[N-1]
Polyline
  name=text
  N=integer // number of points
  point=(double,double,double) // (x,y,z) coordinate in m
  ...
  point=(double,double,double) // (x,y,z) coordinate in m
  normal=(double,double,double) // normal to the polyline (i.e. normal to the drawing plane)
EndPolyline

Polycircle
  name=text
  N=integer // number of vertices
  center=(double,double,double) // (x,y,z) coordinate of the center in m
  point=(double,double,double) // (x,y,z) coordinate of the first point on the circle in m
  normal=(double,double,double) // normal to the polycircle (i.e. normal to the drawing plane)
EndPolycircle

Rectangle
  name=text
  origin=(double,double,double) // (x,y,z) coordinate in m
  width=double // width in m
  height=double // height in m
  u=(double,double,double) // (x,y,z) vector direction for u
  v=(double,double,double) // (x,y,z) vector direction for v
  normal=(double,double,double) // normal to the rectangle (i.e. normal to the drawing plane)
EndRectangle

// 3D operations

// extrude requires one 2D primitive child
Extrude
  name=text
  [material=text] // valid if not nested
  length=double; // extrusion length in m

  Item // 2D primitive polyline (closed), polycircle, or rectangle
  ...
  EndItem
EndExtrude
```

```

// merge requires two children
Merge
  name=text
  [material=text] // valid if not nested

  Item          // extrude, merge, subtract, or SOLID or COMPOUND Brep; implies nesting
  ...
  EndItem

  Item          // extrude, merge, subtract, or SOLID or COMPOUND Brep; implies nesting
  ...
  EndItem
EndMerge

// subtract requires two children
Subtract
  name=text
  [material=text] // valid if not nested

  Item          // extrude, merge, subtract, or SOLID or COMPOUND Brep; implies nesting
  ...
  EndItem

  Item          // extrude, merge, subtract, or SOLID or COMPOUND Brep; implies nesting
  ...
  EndItem
EndSubtract

BRep
  name=text
  [material=text] // valid if not nested

  text          // definining the Brep
EndBRep

// any number of additional items

Item
  ...
EndItem

...

Item
  ...
EndItem

```

References

- [1] C. Geuzaine and J.-F. Remacle, "Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities," *International Journal for Numerical Methods in Engineering*, 79(11), pp. 1309-1331, 2009.
- [2] <https://gmsh.info/> version 4.13
- [3] <https://www.libreoffice.org/>
- [4] <https://www.paraview.org/>
- [5] <https://www.freecad.org/> version 0.21